

AN2DL - First Challenge Report

I Neuron Ribelli

Alberto Ondeï, Andrea Valentini, Abdullah Javed, Filippo Barbari

albertoondei, andreval69, javedhercules, filippobarbari

??????, ??????, ??????, ??????

November 18, 2025

1 Introduction

The goal of this challenge was to predict the **true pain level** of each subject from multivariate time-series data. Our work focused on three main objectives:

- **Identify** the roles of static features vs dynamic joint signals;
- **Model** temporal patterns effectively, avoiding overfitting;
- **Address** the strong imbalance between the three pain classes.

We adopted a step-by-step approach:

1. Initial EDA and establishment of a **baseline model** separating static and dynamic inputs (in Notebook 1);
2. Notebook 2 focuses on the introduction of **temporal feature engineering** to enrich joint dynamics and provide summary statistics (non-learnable features) to the model;
3. Notebook 3 shows the design and tuning of a **final architecture** combining Temporal CNN, BiLSTM with attention, and static feature fusion, plus balanced training and calibration.

2 Problem Analysis

Dataset structure

Each subject corresponds to a 160-step time-series, with:

- 4 pain surveys,
- 30 dynamic joints + 1 static joint,
- 3 static pirate traits (as strings, leg, hand and eye counts).

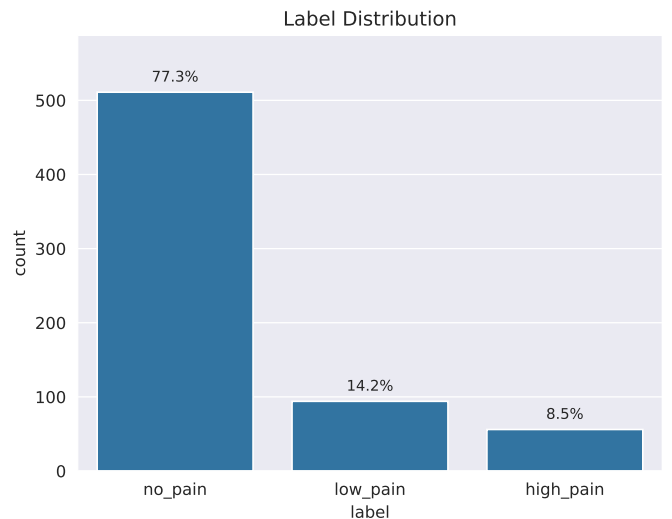


Figure 1: Label distribution is strongly imbalanced toward **no_pain**. This motivated the use of macro F1 and class balancing techniques.

Initial attempts

An initial baseline on raw sequences highlighted two issues:

- strong bias toward `no_pain`;
- unstable performance on minority classes.

This motivated explicit temporal feature engineering and a clearer separation between static and dynamic inputs.

3 Method

Feature representation

Pirate traits (strings) are parsed into:

- numeric counts of legs, hands and eyes;
- binary flags (peg leg, hook hand, eye patch).

Dynamic features:

- surveys (ordinal, not scaled);
- joints (scaled continuous values).

Final shapes:

- dynamic tensor: (661, 160, 34);
- static vector: (661, 7).

Temporal feature engineering

To improve the quality of the temporal signal before feeding it to the model, we expand each of the 30 dynamic joints into three informative channels: (i) the raw measurement, (ii) its first temporal difference (a proxy for local velocity), and (iii) a running standard deviation computed over a nine-step window, which captures short-term instability and local noise patterns. This yields a total of 90 numerical channels per time step.

Since it is difficult for a model to reconstruct many temporal properties (global trends, amplitude variability, and the timing of key events) from short sequences, we also provide sequence-level summary statistics for every numeric feature. These include distributional descriptors (mean, standard deviation, percentiles, and range); trend indicators (difference between the first and last samples, ordinary least squares (OLS) slope, and total variation); temporal shape cues (location of maxima and zero-crossings); and coarse time-region aggregates (statistics computed over early, mid, and late portions of the sequence). All summaries are concatenated into a fixed-size vector, standardized, and used as an additional static input. This compact representation provides the model with an overview of each joint’s global behavior, reducing the burden

of extracting long-term patterns from raw sequences alone.

Model architecture: PainTCNBiLSTMAttn

The model combines different architectures to effectively capture temporal patterns in the data while integrating static features. The main components are:

1. **Embedding Layer:** each of the 4 survey inputs is passed through its own embedding layer to create dense representations.
2. **Concatenation:** the numerical features and the survey embeddings are concatenated along the feature dimension.
3. **Projection Layer:** the concatenated features are projected to the TCN channel dimension.
4. **TCN Layer:** the projected features are passed through the Temporal Convolutional Network (TCN).
5. **SE Block:** the output of the TCN is refined using a Squeeze-and-Excitation (SE) block.
6. **BiLSTM Layer:** the refined features are processed by a Bidirectional LSTM to capture temporal dependencies.
7. **Attention Pooling:** attention pooling is applied to obtain a fixed-size dynamic feature vector.
8. **Static MLP:** the static features are processed through a Static Multi-Layer Perceptron (MLP).
9. **Summary MLP:** the summary features are processed through a Summary MLP.
10. **Gating Mechanism:** a gating mechanism modulates the dynamic feature vector based on the static and summary features.
11. **Final Classification Head:** the concatenated dynamic, static, and summary features are passed through the final classification head to produce logits.

We used a BiLSTM after TCN to capture longer-term dependencies in sequential data. The bidirectional configuration allows the LSTM to access information from both past and future contexts simultaneously. We used only one layer of LSTM because the preceding TCN already serves as a strong feature extractor. We did not use dropout in the LSTM as the preceding TCN already includes dropout for regularization.

Training

The training phase used an extensive hyperparameter search via stratified group 5-fold cross-validation to identify the best configuration for the PainTCNBiLSTMattn model. This process was carried out in two main phases:

- A. In this phase, we searched for the optimal class-2 multiplier, batch size, dropout rate, and model width by evaluating 288 configurations using 5-fold CV.
- B. The best hyperparameter combination identified in Phase A was used to train the model again using stratified group 5-fold CV. Test-Time Augmentation (TTA)[3] was applied during inference to improve robustness, and temperature scaling along with prior correction were employed to enhance calibration and performance on the test set.

As a result, we obtained a well-calibrated and robust model capable of accurately predicting pain levels from the provided data, composed of:

- A weighted random sampler to address class imbalance during training, with a class-2 multiplier of 0.7 (balanced emphasis on class 2).
- A batch size of 16 to ensure stable training.
- A dropout rate of 0.4 to prevent overfitting.
- A model width of 32 for both TCN channels and LSTM hidden size, balancing model capacity and computational efficiency.

4 Experiments and Final Results

One of the main focuses of our experiments was to find a stable configuration that balanced performance and variance across folds. During hyperparameter tuning, we assigned different weights to the second class (`low_pain`) to address the class imbalance. We experimented with multipliers ranging from 0.3 to 1, and the research phase concluded with an extensive grid search comprising approximately 288 configuration combinations and 5-fold cross-validation (CV), equating to 1,440 training runs ($288 \times 5 = 1,440$ training runs, ≈ 34 hours), in order to identify the optimal class-2 multiplier, batch size, dropout rate and model width.

To ensure robustness, we also evaluated each configuration using cross-validation, focusing on the mean and standard deviation of the macro F1 score

across folds. We employed different techniques to enhance model stability, including Exponential Moving Average (EMA) of model weights during training [2], temperature scaling for calibration [1] and light Test-Time Augmentation (TTA, small noise addition and time shifts)[3].

Mean F1 Score vs Standard Deviation of F1 Score (All Configurations)



Figure 2: Although cross-validation yields a mean macro-F1 of 0.871 (std 0.036), our final **Kaggle submission scores 0.94**. This gap is expected and arises because cross-validation evaluates a model trained on only 80% of the data and without test-time enhancements. The final model used for submission is trained on 100% of the training set and benefits from calibration (temperature scaling, logit bias) and Test-Time Augmentation (TTA), all of which are not included in the cross-validation score. Therefore, the CV score is a conservative estimate, while the Kaggle score reflects the full model capacity and improved inference pipeline.

5 Discussion and Conclusions

We combined temporal feature engineering and a hybrid TCN-BiLSTM-Attention architecture to address the Pirate Pain sequence classification task. Cross-validation produced a macro-F1 of 0.871 (std 0.036), confirming the stability of the selected configuration.

The final model, trained on the full dataset and enhanced with calibration and Test-Time Augmentation, achieved **0.9466 macro-F1** on the test set. This improvement is expected since CV models see only 80% of the data and exclude inference-time refinements.

Given more time, potential extensions could include testing simpler baselines as additional ensemble members or trying alternative attention mechanisms.

References

- [1] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 06–11 Aug 2017.
- [2] Daniel Morales-Brotons, Thijs Vogels, and Hadrien Hendrikx. Exponential moving average of weights in deep learning: Dynamics and benefits. *arXiv preprint arXiv:2411.18704*, 2024.
- [3] Guotai Wang, Wenqi Li, Sébastien Ourselin, and Tom Vercauteren. Automatic brain tumor segmentation using convolutional neural networks with test-time augmentation. In *International MICCAI Brainlesion Workshop*, pages 61–72. Springer, 2018.